

Introduction to Web Application Attacks

Unit 10 — How the Web Works, Burp Suite & the OWASP Top 10

Almost everything you use online is a web application. This unit opens the hood.

What we'll do this week

- **Day 1:** How the web talks — requests, responses, methods, status codes
- **Day 2:** URLs, parameters, cookies & sessions, client vs server
- **Day 3:** View Source, Developer Tools, and the OWASP Top 10
- **Day 4:** Burp Suite — proxy and intercept
- **Day 5:** Burp Repeater + content discovery + finish the lab

Learning objectives

By the end of this unit you can:

- **Describe** the request/response model and name the client and server roles.
- **Identify** the parts of an HTTP request and response.
- **Match** common methods (GET/POST) and status codes (200/301/403/404/500).
- **Break down** a URL into its parts.
- **Explain** client-side vs server-side, with an example of each.

Learning objectives (continued)

- **Use** View Source and Developer Tools to inspect HTML, requests, cookies, headers.
- **Explain** what cookies and sessions are and why they matter for security.
- **List** the OWASP Top 10 categories at an awareness level.
- **Configure** Burp Suite as a proxy and **intercept, read, and modify** a request.
- **Use** Burp **Repeater** to resend and tweak a request.
- **Explain content discovery** and why hidden paths matter.

Vocabulary — the web's vocabulary (1 of 3)

Term	Meaning
Web application	A program you use through a browser; most work happens on a remote server.
Client	The side that makes the request — usually your browser.
Server	The side that receives the request and sends back a response.
HTTP	The rules browsers and servers use to talk; the language of the web.
HTTPS	HTTP wrapped in encryption (TLS) so others can't read or change traffic.

Vocabulary (2 of 3)

Term	Meaning
Request	The message a client sends asking for something.
Response	The message a server sends back, with a status code and content.
HTTP method	The "verb" of a request — GET (fetch), POST (send data).
Status code	A 3-digit number saying how the request went.
Header	A <code>name: value</code> line carrying extra info.
Body	The main content of a request or response.

Vocabulary (3 of 3)

Term	Meaning
URL	The full address of a resource.
Query string	The part after <code>?</code> that holds parameters.
Parameter	A <code>name=value</code> pair of input.
Cookie	Small data the server has your browser store and send back.
Session	How the server remembers who you are, often via a cookie.
Proxy	Software between browser and server that can see/change traffic.
OWASP Top 10	The ten most critical web application security risks.

Day 1

How the web works: requests, responses, methods, status codes

Warm-up: what really happens?

When you type a URL and press Enter:

- Your **browser (client)** sends a **request** to a **server**.
- The server does some work and sends back a **response**.
- That back-and-forth is the whole web — every page, every click.

The web is a conversation: client asks, server answers.

The request / response model

```
[ Your Browser ]   --- request ---> [ Web Server ]  
    (client)                (server)  
[ Your Browser ] <--- response --- [ Web Server ]
```

- The **client** starts every conversation.
- The **server** waits, receives the request, and replies.
- One page load is usually **many** requests (HTML, then CSS, JS, images).

Analogy: the web is like a restaurant

- **You (client)** read the menu and place an **order** (request).
- **The kitchen (server)** cooks and sends out a **plate** (response).
- You never walk into the kitchen — you only see what comes out.

The kitchen never serves food you didn't order — the **client asks first, every time.**

Why this matters

- Every login, every game move, every post is a **request** going somewhere.
- If you can **read** the request, you understand exactly what the app is doing.
- If you can **change** it, you can test whether the app trusts you too much.
- Attackers and defenders both start here — by reading the conversation.

You can't attack — or defend — a conversation you can't read.

Anatomy of an HTTP request

```
GET /search?q=cats HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Cookie: session=abc123
```

Four parts:

- **Method** — the verb (`GET`)
- **Path** — what you want (`/search?q=cats`)
- **Headers** — extra info (`Host` , `User-Agent` , `Cookie`)
- **Body** — main content (empty for most GETs)

Anatomy of an HTTP response

```
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: session=abc123

<html>
  <body>Here are your cat results...</body>
</html>
```

Three parts:

- **Status code** — how it went (200 OK)
- **Headers** — info about the response
- **Body** — the actual content (the HTML page)

Worked example: one full round trip

You search for cats. Your browser sends:

```
GET /search?q=cats HTTP/1.1  
Host: example.com
```

The server answers:

```
HTTP/1.1 200 OK  
Content-Type: text/html
```

One request, one response. The `200` says "found it"; the body holds the page.

15

Headers carry the extra info

```
Host: example.com  
User-Agent: Mozilla/5.0  
Accept: text/html  
Cookie: session=abc123
```

- **Host** — which site you want (one server hosts many sites).
- **User-Agent** — what browser you're using.
- **Cookie** — who you are (your session ticket).

Headers are the request's "fine print" — small lines that change everything.

Check your understanding

A request starts with `POST /login HTTP/1.1`.

What are the method and the path?

Answer

- **Method:** `POST` — sending data (a login).
- **Path:** `/login` — the resource being acted on.

The first word is always the **method**; the next is the **path**.

GET vs POST

	GET	POST
Purpose	Fetch a resource	Send data (logins, forms)
Where params go	In the URL / query string	In the body
Visible in address bar?	Yes	No
Shareable link?	Yes	No

Key idea: **both** GET and POST can be read and changed in a proxy. Visibility is **not** security.

Status code families

Family	Meaning	Examples
2xx	Success	200 OK
3xx	Redirect	301 Moved, 302 Found
4xx	Client error	403 Forbidden, 404 Not Found
5xx	Server error	500 Internal Server Error

The ones that matter to us: **200** (here it is), **301/302** (look elsewhere), **403** (exists, but forbidden), **404** (nothing here), **500** (server crashed).

4xx = "you messed up", 5xx = "the server messed up." And to an attacker, **403 and 200 both mean "something is there."**

Status codes tell a story

- A `200` after a login → it worked.
- A `302` after a login → you're being redirected (usually to a dashboard).
- A `403` on a page → you're not allowed (but it exists!).
- A `500` after odd input → you may have **broken something** worth probing.

Testers read status codes like a detective reads a witness — every number is a clue.

Check your understanding

You request `/admin` and get back **403 Forbidden**.
Does the `/admin` page exist?

Answer

- **Yes — it exists.** `403` means "this is real, but you're not allowed in."
- A `404` would mean "nothing here at all."
- To a tester, a `403` is a **confirmed target** worth investigating.

403 = "it's there, just locked." That's a clue, not a dead end.

Day 1 guided practice + exit ticket

Using the **Anatomy of HTTP** handout:

1. Label a printed request (method, path, headers, body) and response (status, headers, body).
2. Decode a list of status codes — what does each tell you?

Lab start: Begin TryHackMe "Burp Suite: The Basics" intro tasks (concepts only — no interception yet).

Exit ticket: *What does status code **404** mean, and is it the client's or the server's fault?*

Day 2

URLs, parameters, cookies & sessions, client vs server

Anatomy of a URL

```
https://shop.com/search?item=shoes&size=9
```

scheme host path query string

- **Scheme** — `https` (the protocol)
- **Host** — `shop.com` (the server)
- **Path** — `/search` (the resource)
- **Query string** — everything after `?`

Parameters live in the query string

```
?item=shoes&size=9
```

- Each `name=value` pair is a **parameter**.
- `&` separates multiple parameters.
- `item=shoes` and `size=9` are **input** sent to the server.

Parameters are user input — and user input is exactly what attackers learn to abuse in Units 11-12.

Worked example: read the parameters

```
https://shop.com/product?id=42&color=red&qty=2
```

- **id** = 42 — which product
- **color** = red — a choice
- **qty** = 2 — how many

What happens if you change `qty=2` to `qty=-1`, or `id=42` to `id=43`? That question is the whole game.

Check your understanding

In `https://site.com/news?topic=sports&page=3`,
how many parameters are there, and what are their names?

Answer

- **Two parameters:** `topic` and `page`.
- Values: `topic=sports`, `page=3`.
- The `&` separates them; everything after `?` is the query string.

Count the `name=value` pairs — each one is a parameter you could change.

What is a cookie?

- A **small piece of data** the server tells your browser to store.
- The browser **sends it back** on every later request to that site.
- Set by the server with a `Set-Cookie` header:

```
Set-Cookie: session=abc123; HttpOnly; Secure
```

Cookies are how a site "remembers" you between page loads.

Cookies and sessions

- HTTP is **stateless** — each request stands alone; the server forgets you instantly.
- A **session** fixes that: the server stores who you are and gives the browser a **session cookie** as a ticket.
- Steal that cookie → impersonate the user. That's why cookies are a security target.

Two protective flags:

- `HttpOnly` — JavaScript can't read the cookie (blocks many XSS theft attempts).
- `Secure` — cookie only sent over HTTPS.

Analogy: a session cookie is a coat-check ticket

- You hand in your coat, get a **numbered ticket** (the cookie).
- The ticket doesn't say your name — but whoever holds it gets the coat.
- **Steal the ticket → walk off with someone else's coat.**

That's why a stolen session cookie = a stolen identity. The site trusts the ticket, not the person.

Check your understanding

A cookie is set with the `HttpOnly` flag.

Can a page's JavaScript read that cookie's value?

Answer

- **No.** `HttpOnly` hides the cookie from JavaScript entirely.
- This blocks a common XSS trick: a script that tries to steal `document.cookie`.
- The cookie still rides along on requests — the browser sends it, JS just can't see it.

`HttpOnly` is a key defense we'll revisit when we study XSS in Unit 11.

Client-side vs server-side

	Client-side	Server-side
Runs where?	In your browser	On the server
Examples	HTML, JavaScript	Database, business logic
Can the user see/change it?	Yes	No

Anything client-side is under the **user's** control.

The #1 security rule of the web

Never trust the client.

- JavaScript form validation runs in **your** browser — you can disable or bypass it.
- A proxy (Burp) lets you edit a request **after** the JavaScript "checked" it.
- Real security checks must run **server-side**, where the user can't tamper.

Day 2 guided practice + exit ticket

1. Take a sample URL apart into scheme, host, path, query string, parameters.
2. For a login form, label client-side vs server-side:
 - Typing in the box → client-side
 - "Password too short" popup → client-side (bypassable!)
 - Checking the password against the database → server-side

Lab: continue the TryHackMe Burp room.

Exit ticket: *Give one example of something that happens client-side and one server-side.*

Day 3

View Source, Developer Tools & the OWASP Top 10

View Source vs Developer Tools

- **View Source** (`Ctrl+U`) — shows the **raw HTML** the server sent. Static snapshot.
- **Developer Tools** (`F12`) — a live, interactive toolkit:
 - **Elements** — the live HTML/DOM
 - **Network** — every request and response
 - **Storage / Cookies** — what the site stored on you

Secrets hide in plain sight

```
<!-- TODO: remove before launch. admin pw = hunter2 -->  
<input type="hidden" name="price" value="9.99">  
<input type="hidden" name="role" value="user">
```

- Developers leave **comments** in HTML — visible to anyone.
- **Hidden fields** aren't hidden from a proxy — change `price` or `role` and resend.

"Hidden" in HTML means hidden from the eye, **not** from the attacker.

Check your understanding

```
<input type="hidden" name="price" value="9.99">
```

A checkout page sends this hidden field. **Can a shopper change 9.99 before it reaches the server?**

Answer

- **Yes.** "Hidden" only hides it from the page — not from a proxy.
- In Burp, the shopper can intercept the request and edit `price` to `0.01`.
- The fix: the **server** must look up the real price, never trust the field.

Never trust the client. The price must be decided server-side.

Reading a request in the Network tab

1. Open Dev Tools → **Network** tab.
2. Reload the page or click a link.
3. Click any request to see:
 - **Headers** (request + response)
 - **Method** and **status code**
 - **Payload** (parameters sent)
 - **Cookies**

What is OWASP?

- **OWASP** = Open Worldwide Application Security Project.
- A **nonprofit** that publishes **free** web-security guidance.
- Most famous output: the **OWASP Top 10** — the ten most critical web app risks.

The Top 10 is the **map** of what we'll study in Units 11 and 12.

OWASP Top 10 (awareness) — part 1

#	Category	One line
A01	Broken Access Control	Users do things they shouldn't be allowed to.
A02	Cryptographic Failures	Weak/missing encryption exposes data.
A03	Injection	Untrusted input runs as code (SQLi, XSS).
A04	Insecure Design	The flaw is baked into the design.
A05	Security Misconfiguration	Bad defaults, exposed settings.

OWASP Top 10 (awareness) — part 2

#	Category	One line
A06	Vulnerable Components	Outdated libraries with known holes.
A07	Auth Failures	Weak logins, broken sessions.
A08	Integrity Failures	Untrusted updates/data trusted blindly.
A09	Logging & Monitoring Failures	Attacks go unnoticed.
A10	SSRF	The server is tricked into making requests.

We zoom into **A03 Injection** in Units 11 (XSS, command injection) and 12 (SQLi).

Why the Top 10 matters

- It's the security world's **shared language** — everyone knows what "A03" means.
- Bug-bounty reports, pentest findings, and job interviews all reference it.
- It's a **checklist**: build an app, then ask "did I avoid all ten?"

Knowing the map first means the attacks in Units 11–12 won't feel random.

Check your understanding

An app lets a normal user open another user's private messages just by changing a number in the URL.

Which OWASP category is this?

Answer

- **A01 — Broken Access Control.**
- The user is doing something they shouldn't be allowed to do.
- The fix: the **server** must check "are *you* allowed to see this?" on every request.

Changing an `id` to reach someone else's data is the classic Broken Access Control bug.

Day 3 guided practice

On an **approved practice page**:

1. Open **View Source** — find an HTML comment.
2. Open Dev Tools → **Elements** — find a hidden form field.
3. Open the **Network** tab — inspect one request (method, status, headers).

Lab: begin the TryHackMe "OWASP Top 10" room intro tasks (awareness).

Day 3 exit ticket

Name **two** OWASP Top 10 categories and write **one sentence** about each.

Day 4

Burp Suite: proxy & intercept



Authorization — read before you touch Burp

A proxy lets you **see and modify** real traffic — including data a site never meant you to edit. That power is exactly where "testing" becomes a **crime** without permission.

- Targets are **intentionally vulnerable practice apps ONLY**: TryHackMe rooms, PortSwigger Web Security Academy, DVWA.
- ❌ Never the school portal. ❌ Never a bank. ❌ Never **any** real site — even "just to look."
- Unauthorized use violates computer-crime law (CFAA). **Minors are not exempt.**

Authorization and scope are the only line between a tester and a criminal.

What is an intercepting proxy?

- A **proxy** sits **between** your browser and the server.
- An **intercepting** proxy can **pause** a request so you can read or edit it before it goes on.

```
[ Browser ] → [ Burp Proxy ] → [ Server ]  
                (read / pause / edit here)
```

Burp sits in the middle of the conversation and can change what's said.

Analogy: pausing the waiter

- Remember the restaurant? The **waiter** carries your order to the kitchen.
- An intercepting proxy is like **freezing the waiter mid-step**.
- You read the order slip, maybe **change** it, then say "go" — the kitchen never knows.

Burp is the frozen waiter: you control the order before the kitchen sees it.

Burp Suite at a glance

- **Burp Suite Community Edition** — free toolkit from PortSwigger.
- Key tabs we'll use:
 - **Proxy** — intercept and view traffic
 - **HTTP history** — a log of every request/response
 - **Repeater** — resend and edit one request (Day 5)

The **embedded browser** (Proxy → Open Browser) is pre-configured — use it to skip proxy/certificate setup.

The intercept workflow

1. **Proxy → Intercept** → toggle "**Intercept is on.**"
2. In the Burp browser, click a link or submit a form.
3. The browser **appears to hang** — that's normal! The request is **paused at Burp**.
4. Read the paused request (method, path, headers, body).
5. Click **Forward** to let it continue.

A page load is many requests — you may click **Forward** several times.

Demo: intercept and forward

```
GET /vulnerabilities/xss_r/?name=Aaron HTTP/1.1  
Host: 10.10.10.5  
Cookie: PHPSESSID=...; security=low
```

- Turn Intercept **on**, trigger this request.
- Read it in the Intercept tab.
- Click **Forward** — watch it appear in **HTTP history**.
- Turn Intercept **off** when done so browsing works normally.

Day 4 guided practice & lab start

1. Read the **Safety & authorization reminder** in the lab aloud with your partner.
2. Write your scope statement at the top of your journal:
"I am authorized to test only this practice app provided by my instructor."
3. Set up Burp (embedded browser **or** AttackBox).
4. Confirm traffic flows through Burp → **HTTP history**.
5. Intercept your **first** request.

Lab Steps 1-3 follow (next slides).

Lab Step 1 — Prove traffic flows through Burp

- With Intercept **off**, browse the practice app in the Burp browser.
- Open **Proxy → HTTP history** — you should see your requests listed.
- **Record:** pick one GET request. Write its **method**, **path**, one **header**, and the **status code**.

Expected: a normal page load returns `200 OK`.

Lab Step 2 — Intercept your first request

- Turn **Intercept ON**.
- Click a link or submit a form in the Burp browser.
- The browser "hangs" — the request is **paused at Burp**.
- Identify the **method**, **path**, **headers**, and **body** (if a form).
- Click **Forward** until the page loads.
- **Record:** method, path, and body contents.

Lab Step 3 — Modify a request before forwarding

- Find a request with a **parameter** you can change (e.g., `?q=cats`).
- Intercept it, then **edit a value** directly (`q=cats` → `q=dogs`, or a hidden `price` / `id`).
- Click **Forward** and watch the page.
- **Record:** what you changed (from → to) and how the response differed.

Did the change "work"? That tells you whether the check is **client-side** (bypassable) or **server-side**.

Day 4 exit ticket

Where does **Burp** sit in the conversation between your browser and the server?

Check your understanding

You turn Intercept **on**, click a link, and the browser seems to **freeze**.
Is something broken? What do you do?

Answer

- **Nothing is broken.** The request is **paused at Burp**, waiting for you.
- Read it, then click **Forward** to let it continue.
- A page is many requests — you may Forward several times.
- Done testing? Turn Intercept **off** so browsing works normally.

The "frozen browser" is the #1 Day 4 surprise. It's just waiting.

Day 5

Burp Repeater + content discovery + finish the lab

Burp Repeater

- **Repeater** = edit and resend **one** request, over and over, fast.
- No need to redo the whole browser action each time.
- The tester's **workhorse** for probing how a server reacts to changes.

Workflow:

1. Right-click a request in HTTP history → **Send to Repeater**.
2. Click **Send**, read the response.
3. **Edit** something, **Send** again, compare.

Lab Step 4 — Send to Repeater and experiment

```
GET /vulnerabilities/sqli/?id=1 HTTP/1.1    ← original
GET /vulnerabilities/sqli/?id=2 HTTP/1.1    ← variation 1
POST /vulnerabilities/sqli/      HTTP/1.1    ← variation 2 (method swap)
```

- Send the request to Repeater, click **Send**.
- Edit a parameter / header / method, **Send** again.
- **Record:** at least **two** variations — what you changed and how the response differed.

Content discovery — the concept

- Tools like **gobuster** and **dirb** request many **guessed** paths from a wordlist.
- They read the **status code** of each guess to find hidden pages.

```
gobuster dir -u http://TARGET -w /usr/share/wordlists/dirb/common.txt
```

Guess	Status	Meaning
/admin	403	Exists, but forbidden
/backup	200	Exists and readable!
/xyzyz	404	Nothing there

Why 403 is a gift to an attacker

- **404 Not Found** → there's nothing there. Dead end.
- **403 Forbidden** → the resource **exists** but access is denied.
- **200 OK** → it exists and you can read it.

A **403** is useful because it **confirms a hidden path is really there** — now the attacker knows where to dig.

Lab Step 5 — Content discovery (concept)

With your partner, explain in 2–3 sentences:

Why is a **403 Forbidden** actually a **useful** result to an attacker doing content discovery?

Optional stretch: run real content discovery against an approved practice target and interpret the status codes you get back.

Lab deliverables

Intercepted / Modified Request Log:

#	Method	Path/URL	Changed (from → to)	Status	What it told me
1	GET	/...	(forwarded)	200	...

Include: 1 intercepted+forwarded (Step 2), 1 modified (Step 3), 2 Repeater variations (Step 4).

Plus a 2–3 sentence reflection linking one observation to an OWASP Top 10 idea.

Day 5 exit ticket

Submit your lab-journal page. In one sentence: **what surprised you about modifying a request?**

Common pitfalls (and fixes)

- **"My browser froze!"** → Intercept is **on**. Click **Forward** or toggle it **off**.
- **HTTPS certificate warnings** → you used the system browser; switch to the **embedded browser** or **AttackBox**.
- **GET vs POST confusion** → both are visible and editable in Burp. Visibility $\neq$ security.
- **Pointed Burp at a real site?** → STOP. Re-read the scope statement.

Recap — the big ideas

- The web is a **request/response** conversation between **client** and **server**.
- **Methods** (GET/POST) and **status codes** tell you what happened.
- **Cookies/sessions** are how a site remembers you — and a security target.
- **Never trust the client** — real checks live **server-side**.
- A **proxy** lets you read and **modify** traffic — only against authorized targets.
- The **OWASP Top 10** is the map for Units 11–12.

Discussion prompt

A student points Burp at their **bank's** login page "just to see what the request looks like" and changes nothing. Is that ethical? Is it legal?

Compare to the Wireshark "I just looked" question from Unit 3 — what's the **same**, and what's **different** now that you can *modify* the request?

Next up

Unit 11: Common Web Attacks — XSS, Command Injection & File Inclusion

We take the OWASP map and start driving: every attack paired with its defense.

Curriculum by Aj Hammond — PNPT, CRT0, OSCP, BSCP

github.com/ajm4n · linkedin.com/in/aj-hammond